

MANUAL TÉCNICO - GOLITE

Intérprete para el lenguaje de programación GoLite, un lenguaje diseñado con una sintaxis inspirada en Go.

Tecnologías utilizadas:

- Java
- JFlex
- Java CUP
- Graphviz
- Swing

Diagrama general de funcionamiento:

Código Fuente

Lexer

Parser

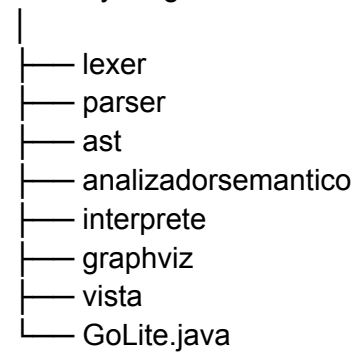
AST

Análisis Semántico

Interpretación

Estructura del proyecto:

com.mynor.golite



Lexer

Responsable del análisis léxico mediante JFlex.

IDENTIFICADOR = [a-zA-Z_][a-zA-Z0-9_]*

ENTERO = [0-9]+

FLOTANTE = [0-9]+."[0-9]+

Parser

Responsable del análisis sintáctico mediante Java CUP.

start with programa;

```
programa ::=  
    lista_globales:l  
    {: RESULT = new NodoPrograma(l, lleft, lright); :}  
    ;
```

AST

Representación interna del programa mediante nodos.

```
public abstract class NodoAST {  
    private final int linea;  
    private final int columna;  
}
```

Analizador Semántico

Implementa las validaciones semánticas.

```
public Tipo visit(NodoIdentificador nodo) {  
  
    Simbolo s = entornoActual.buscar(nodo.getNombre());  
  
    if (s == null) {  
        agregarError(  
            "Variable no existe: " + nodo.getNombre(),  
            nodo.getLinea(),  
            nodo.getColumna()  
        );  
        return new TipoPrimitivo(TipoEnum.ERROR);  
    }  
  
    return s.getTipo();  
}
```

Ejecutor

Ejecuta el programa recorriendo el AST.

```
public Object visit(NodoPrintln nodo) {  
    StringBuilder linea = new StringBuilder();  
    List<NodoExpresion> args = nodo.getArgumentos();  
    for (int i = 0; i < args.size(); i++) {  
        if (i > 0) {  
            linea.append(" ");  
        }  
        linea.append(valorAString(args.get(i).accept(this)));  
    }  
}
```

```
    }  
    salida.append(linea).append("\n");  
    return null;  
}
```

Graphviz

Generación visual del AST.

```
public String visit(NodoContinue nodo) {  
    return nodo("NodoContinue", "#ffcdd2");  
}
```

Análisis Lexico

Descripción:

El análisis léxico es realizado mediante JFlex. Su función consiste en transformar el código fuente en una secuencia de tokens reconocibles por el parser.

Análisis Sintactico

Descripción:

El análisis sintáctico es realizado mediante Java CUP.

Cada producción sintáctica construye directamente nodos del AST.

Visitor

Descripción:

Se utiliza el patrón Visitor para desacoplar las operaciones del AST.

Visitors implementados:

1. Analizador Semántico
2. Ejecutor
3. Manejador Graphviz